

CART: hiperparâmetros e discussão + Algoritmo Random Forest

Aula 06 – Inteligência Computacional

Eduardo Corrêa Gonçalves

09/09/2026

Sumário

Introdução

O que é hiperparâmetro ?

Hiperparâmetros *versus* parâmetros

CART: resumo das dicas do manual da sklearn

Prós e Contras

Dicas de hiperparâmetros para mitigar o overfitting

Random Forest

Intuição

Treinamento e Classificação

Prós e Contras

Hiperparâmetros (1/2)

O que é hiperparâmetro ?

- É um tipo de parâmetro especial **escolhido pelo usuário** de um algoritmo de AM ou IC.
- Os valores dos hiperparâmetros são definidos pelo usuário **antes do treinamento** do modelo.
 - Por exemplo, no **CART** (figura abaixo) → medida de impureza (*criterion*), comprimento máximo da árvore (*max_depth*) etc.
 - Na **Rede Neural** → número de épocas, função de ativação da camada escondida, número de unidades na camada escondida etc.

DecisionTreeClassifier

```
class sklearn.tree.DecisionTreeClassifier(*, criterion='gini',
splitter='best', max_depth=None, min_samples_split=2, min_samples_leaf=1,
min_weight_fraction_leaf=0.0, max_features=None, random_state=None,
max_leaf_nodes=None, min_impurity_decrease=0.0, class_weight=None,
ccp_alpha=0.0, monotonic_cst=None)
```

Hiperparâmetros (2/2)

Qual a diferença entre hiperarâmetro e parâmetro (no vocabulário de AM / IC) ?

- Hiperparâmetros:

- Valores para determinadas variáveis que o **usuário precisa determinar** antes do treinamento.
- O processo de treinamento depende da escolha prévia de tais valores.

- Parâmetros:

- São valores de variáveis que o **modelo aprende sozinho** a partir dos dados de treinamento, ao longo do processo de treinamento
 - **Ex.:** Os pesos w e b de um modelo de regressão logística ou rede neural.
 - O conjunto de parâmetros é denotado por θ .

CART: implementação na sklearn (1/8)

Aula de hoje...

- Na aula de hoje, vamos examinar alguns **hiperparâmetros** da implementação do **CART** na sklearn
 - Especificamente, os que podem ajudar a mitigar o hiperajuste (*overfitting*) do CART aos dados de treino.
- Também vamos apresentar o algoritmo **Random Forest**.
 - Uma evolução do CART, que em muitas aplicações tem desempenho competitivo com o das redes neurais.
- Mas antes, vamos examinar trechos do manual do CART na sklearn.
 - Começando pelos aspectos positivos e negativos do algoritmo.

CART: implementação na sklearn (2/8)

CART – aspectos positivos

1. Algoritmo **muito fácil** de se entender.
2. O modelo (árvore) é **caixa-branca** e pode até mesmo ser visualizado de forma gráfica. Qualquer classificação pode ser explicada por uma regra.
3. Requer **pouca preparação de dados**. Ex.: não é preciso passar as variáveis para a mesma escala, entre outras coisas.
4. Suporte para atributos **nominais e numéricos** (*embora sklearn exija que X seja uma matriz numérica*).
5. **Custo** computacional **baixo**: complexidade logarítmica em relação ao tamanho da base de treinamento.
6. **Bom desempenho**, mesmo quando hipóteses que assume são violadas pelos dados de treino.
7. Serviu de base para o **Random Forest**.

CART: implementação na sklearn (3/8)

CART – aspectos negativos

1. Muito suscetível ao **sobreajuste** (*overfitting*):
 - Pode criar uma AD excessivamente grande que não irá generalizar bem.
 - Em alguns casos a AD gerada é uma “tradução gráfica” do BD de treino.
2. Tende a ser **instável**: uma pequena variação dos dados leva ao aprendizado de uma AD totalmente diferente.
3. A geração de uma **AD ótima** é um problema **NP-completo**.
 - Por isso, o CART usa uma heurística do tipo algoritmo guloso (*greedy*), onde decisões ótimas locais são tomadas para definir cada nó.
4. Pode criar ADs com **viés** se alguma classe for dominante.
5. Não consegue expressar bem alguns conceitos, como XOR (*veremos isso na aula sobre Redes Neurais*)

CART: implementação na sklearn (4/8)

Como mitigar os problemas ?

- **Sobreajuste** (*overfitting*):
 - Configurar alguns hiperparâmetros, como:
 - Número mínimo de exemplos de um nó folha (*min_sample_leafs*)
 - Comprimento máximo da árvore (*max_depth*)
 - Usando o mecanismo de poda.
- **Instabilidade e Heurística Gulosa**
 - Usando o Random Forest
- **Classes Desbalanceadas**
 - Usando um algoritmo para balancear as classes antes do treino, como o SMOTE (*veremos em uma aula futura*)

CART: implementação na sklearn (5/8)

Alguns hiperparâmetros importantes (parte 1)

- **max_depth**: altura máxima da AD (default = None)
 - Se *max_depth* = None, nós serão expandidos até todas as folhas serem puras ou todas as folhas conterem menos de *min_samples_split* observações.
- **min_samples_split**: número mínimo de observações exigidas para dividir um nó interno (default = 2)
 - Se definido como float, valor será = $\text{piso}(\text{min_sample_splits} \times \text{num. observ.})$
- **min_samples_leaf**: número mínimo de observações exigidas em um nó folha (default = 1)
 - Um ponto de divisão será considerado apenas se deixar *min_samples_leaf* observações nos nós que seriam gerados nas ramificações direita e esquerda.
 - Se definido como float, valor será = $\text{piso}(\text{min_sample_leafs} \times \text{num. observ.})$
- **random_state** (default = None): semente para controlar a aleatoriedade.
 - Durante a geração da AD, várias divisões podem gerar o mesmo valor de pureza das partições.
 - Para obter resultado determinístico em todas as rodadas do programa, especifique um valor de semente.

CART: implementação na sklearn (6/8)

Alguns hiperparâmetros importantes (parte 2)

- **max_leaf_nodes** (default = None): gera AD com número máximo de folhas que for definido neste hiperparâmetro.
 - Se *max_leaf_nodes* = None, gera número ilimitado de folhas
 - Caso contrário, os melhores nós são definidos de maneira *best-first*.
- **ccp_alpha** (default = None): parâmetro utilizado para poda da AD.
 - Por padrão, a poda não é executada (*ccp_alpha* = None)
 - Caso contrário, a poda é realizada e a subárvore com maior valor *cost complexity* inferior a *ccp_alpha* será escolhida.
 - Mais detalhes em: <https://scikit-learn.org/stable/modules/tree.html#minimal-cost-complexity-pruning>

CART: implementação na sklearn (7/8)

CART – dicas de uso prático da sklearn (parte 1)

- (1) Começar usando *max_depth* = 3 e visualizar a AD gerada para perceber como ela está se ajustando aos dados de treino.
 - Então, ir incrementando o valor de *max_depth* aos poucos.
- (2) Usar *max_depth* para controlar a altura da AD e mitigar o sobreajuste.
- (3) Usar *min_samples_split* ou *min_samples_leaf* para garantir um número razoável de observações sendo levado em consideração a cada divisão.
 - Um valor muito pequeno eleva o risco de sobreajuste, enquanto um valor grande impedirá a AD de aprender sobre os dados.
 - *min_samples_leaf* = 5 pode ser tentado como um valor inicial
 - Se a quantidade de observações varia muito, um valor float pode ser usado nos 2 parâmetros.
 - Enquanto *min_samples_split* pode criar folhas arbitrariamente pequenas, *min_samples_leaf* garante que cada folha tenha um tamanho mínimo.

CART: implementação na sklearn (8/8)

CART – dicas de uso prático da sklearn (parte 2)

- (4) Em problemas com muitas variáveis, utilizar alguma técnica para reduzir a dimensionalidade
 - **Ex.:** seleção de atributos, PCA
 - Assim, aumentará a chance do CART identificar *features*, de fato, discriminativas
- (5) Tratar o problema do desbalanceamento de classes antes do treino.
 - O objetivo é mitigar o aprendizado de ADs que com viés para a classe predominante.
 - Pode ser feito através da amostragem de um número igual de observações de cada classe.
 - Ou normalizando para um mesmo valor a soma dos pesos dos exemplos (hiperparâmetro *sample_weight* do método **fit**) para cada classe
 - (*a sklearn considera essa forma a preferencial*).

Random Forest (1/12)

Random Forest – Intuição

- Suponha que seu computador esteja travando com frequência.
 - Em vez de chamar apenas um técnico para examiná-lo, você pode optar por chamar vários.
 - Se uma determinada **causa** para o problema for **mais citada** do que qualquer outra, você a consideraria como a diagnóstico correto.
- 
- Ou seja: a causa do problema de travamento seria determinada de acordo com uma espécie de **votação majoritária**, em que cada técnico dá seu voto.
 - Expectativa: considerar a **opinião mais prevalente do grupo** de técnicos é mais confiável do que levar em conta a opinião de um único profissional.

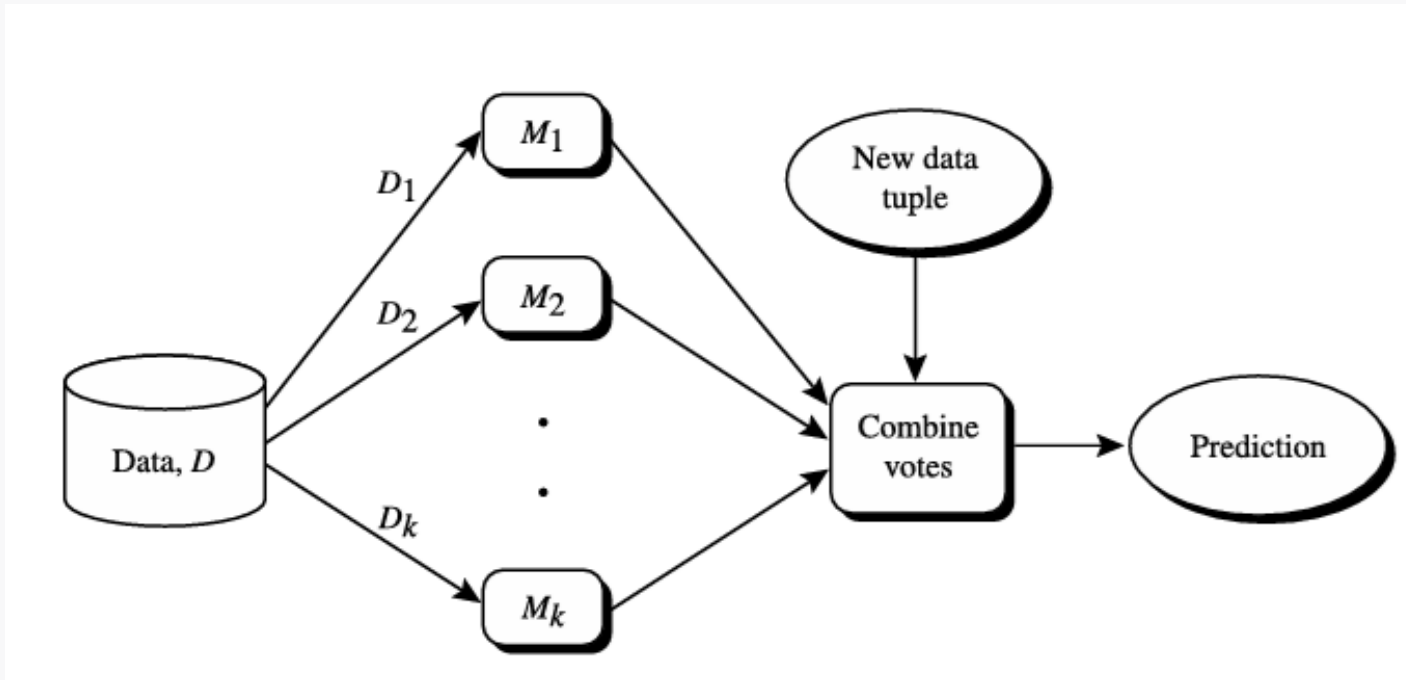
Random Forest (2/12)

Random Forest – Intuição

- A **Floresta Aleatória** (*Random Forest* – RF) é uma técnica de classificação que trabalha com uma ideia similar.
- Ela **combina as decisões de múltiplos modelos** para otimizar a eficácia do processo de classificação.
- Mais especificamente, uma RF é um modelo de classificação composto (*ensemble model*), que é formado por um conjunto (*ensemble*) de classificadores do tipo AD.
- Obs.: Além do RF, há outros métodos de ensemble como o XGBoost, mas não serão cobertos no curso.

Random Forest (3/12)

RF – Esquema de Funcionamento (Han et al. 2023)



- No treinamento, o RF aprende k modelos de classificação $M_1, M_2, \dots, M_k$
- Para classificar um novo objeto (*new data tuple* no diagrama), cada classificador “vota” para a classe do objeto.
- O modelo ensemble combina os votos para retornar a classificação

Random Forest (4/12)

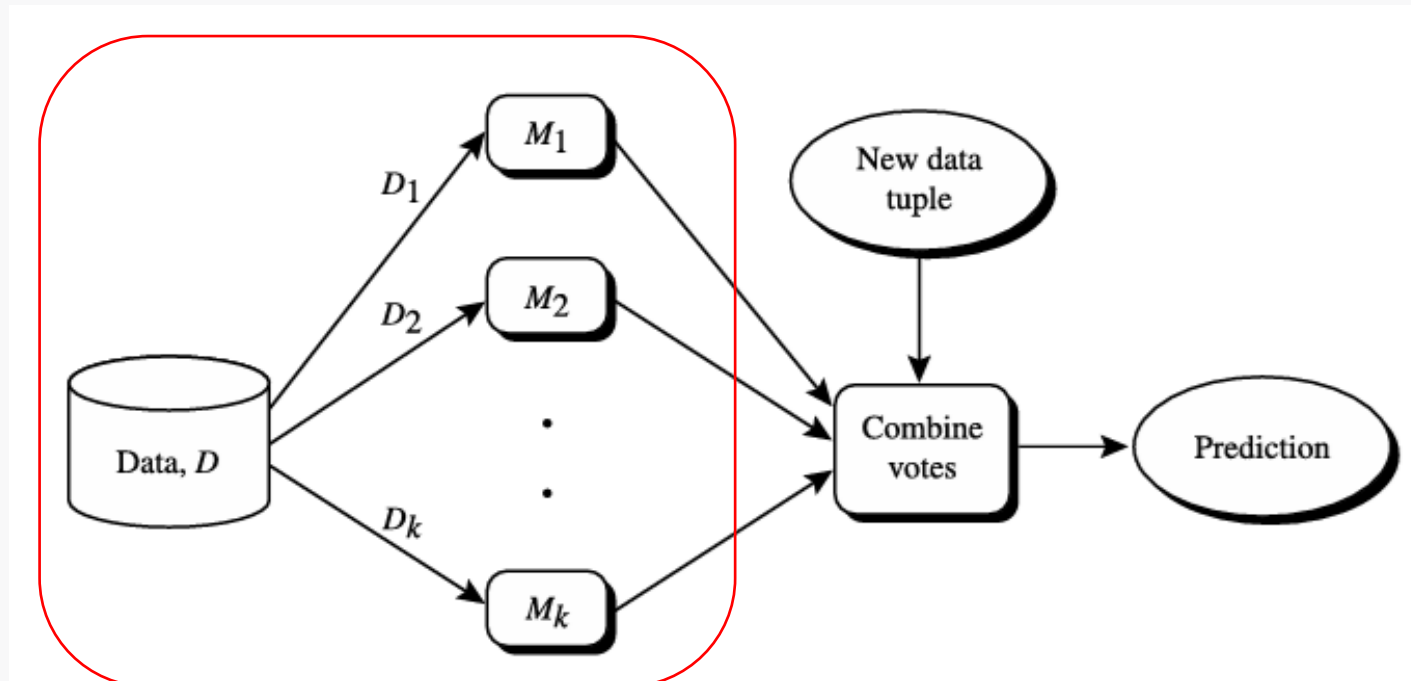
Treinamento de uma RF – Visão Geral

- Um modelo RF é composto por um **grupo de k modelos de árvore de decisão**: $M_1, M_2, \dots, M_k$,
- Cada árvore de decisão individual é também chamada de **classificador base** (*base classifier*).
- Cada classificador base é treinado com o uso de **subconjuntos independentes e aleatórios de atributos** e objetos
- O treinamento das k ADs que compõem a RF é feito em **k iterações**.

Random Forest (5/12)

Treinamento de uma RF – Detalhes

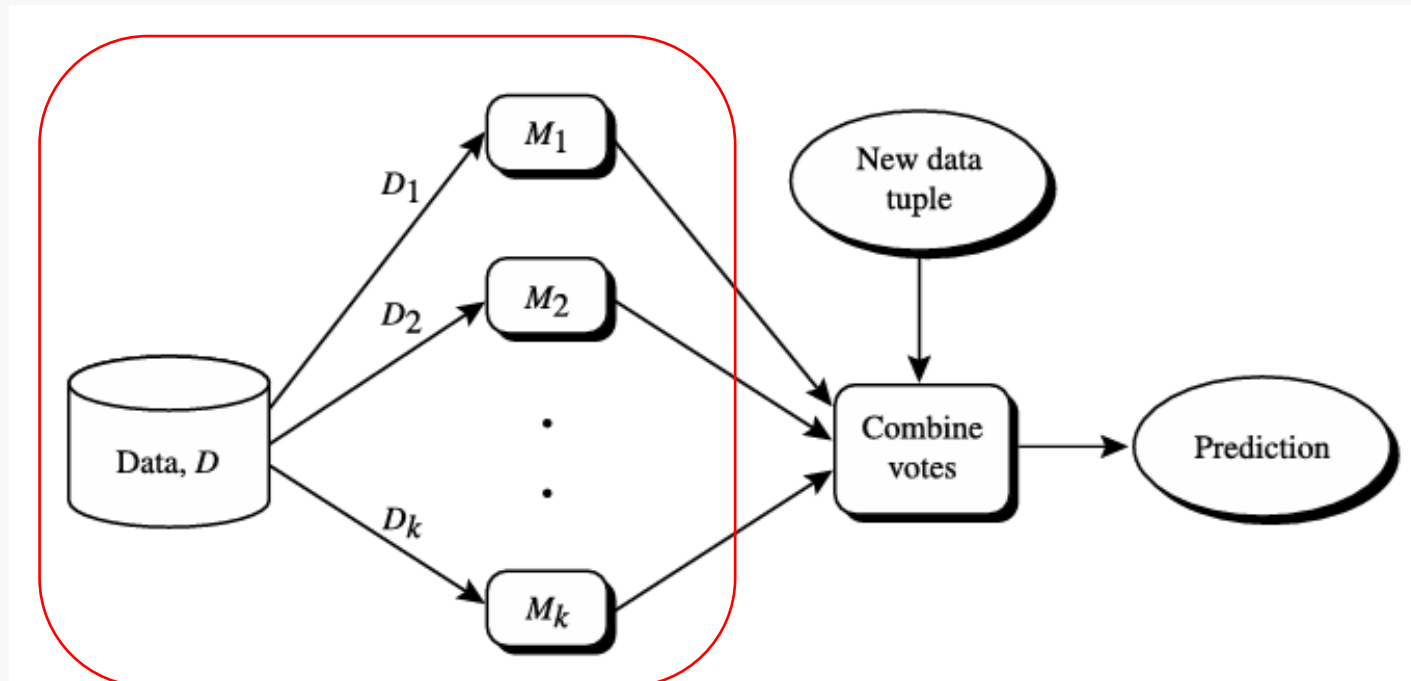
- Treinamento em k iterações, a partir de uma base de treino D .
- Para cada iteração ($i=1, 2, \dots, k$), um conjunto de treinamento D_i , com d objetos é gerado a partir de um processo de amostragem com reposição (*bootstrapping*).
- Cada D_i tem o mesmo número de objetos do que D , porém alguns objetos podem aparecer mais de uma vez, enquanto outros podem não ocorrer em D_i .



Random Forest (6/12)

Treinamento de uma RF – Detalhes (*continuação*)

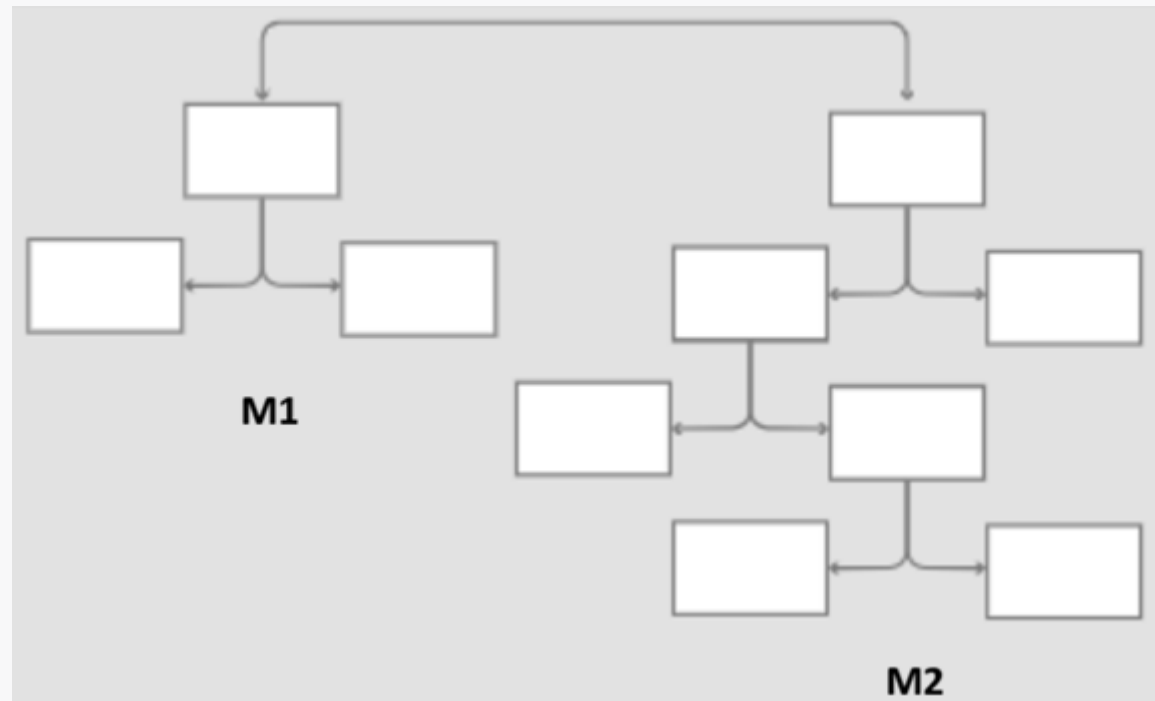
- A cada iteração i , um modelo de árvore de decisão M_i é treinado a partir de D_i com o uso do algoritmo CART.
- As árvores devem ser criadas com tamanho máximo e sem poda.
- Entretanto, a cada nó de uma árvore M_i , apenas um **subconjunto de atributos candidatos** é considerado para decidir qual será a melhor divisão. Esse subconjunto é definido de maneira aleatória.



Random Forest (7/12)

Treinamento de uma RF – Detalhes (*continuação*)

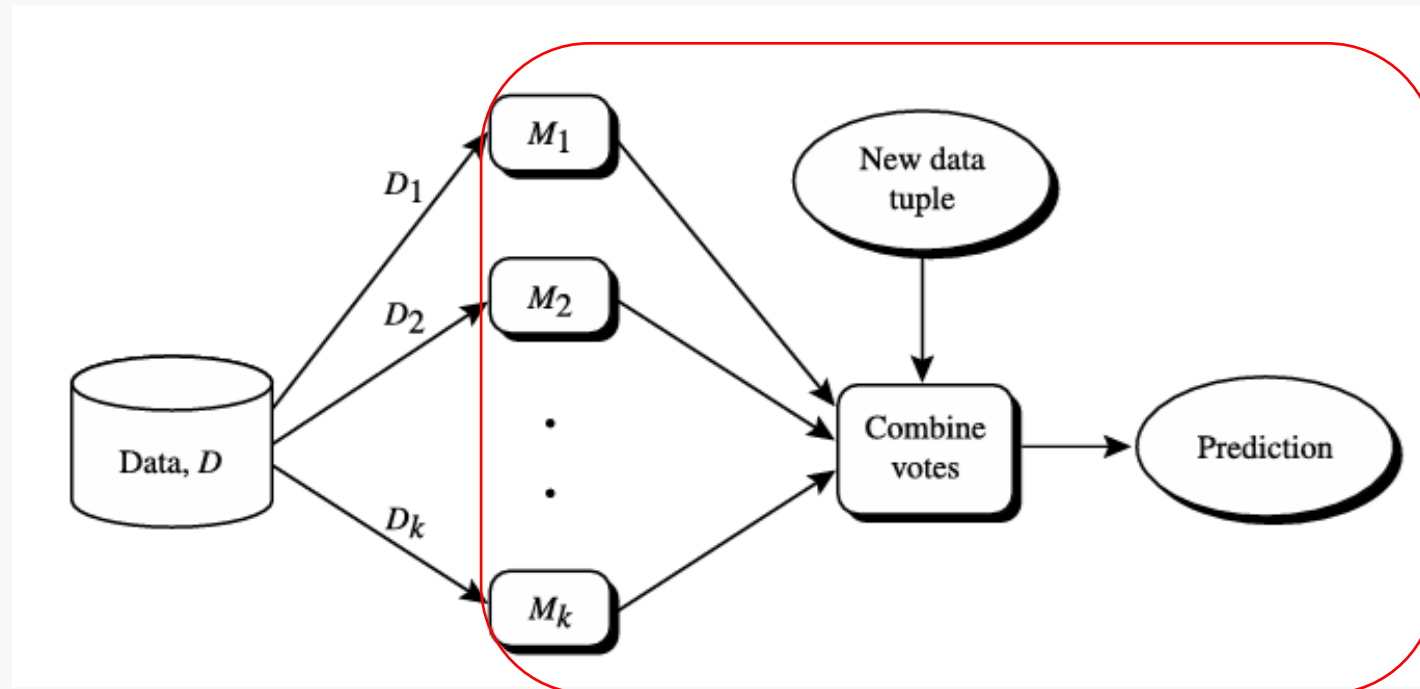
- O modelo final será então composto por diversas ADs
- Elas possuirão estruturas diferentes em termos de quantidade e disposição dos nós, pois cada AD foi treinada a partir de um subconjunto diferente de observações e atributos, sempre determinados de forma independente e aleatória.
- Daí, o nome Floresta Aleatória.



Random Forest (8/12)

Classificação com uma RF

- Uma vez que a RF tenha sido gerada, o processo para classificar um novo objeto é muito simples.
- Durante a classificação, cada AD vota, ou seja, determina individualmente a classe do objeto e a classe mais votada é retornada.
 - *Há outras formas de combinar os votos, mas não abordaremos nessa aula.*



Random Forest (9/12)

RF – aspectos positivos

- Um modelo RF tende a ter um **desempenho preditivo superior** do que cada uma de suas ADs individualmente.
- Os classificadores base (árvores) que compõem a RF até poderão cometer erros de classificação, porém a RF propriamente dita só **errará a classe** de um novo objeto quando **mais da metade se equivocarem**.
- Estudos demonstram que, em diversos domínios de aplicação, as RFs possuem desempenho preditivo tão bom quanto o de métodos mais complexos e caros, como as redes neurais profundas.
 - As **RFs são eficientes** em bases muito volumosas, pois consideram um número reduzido de atributos candidatos para gerar cada nó.
 - As etapas de treinamento e classificação da RF são facilmente paralelizáveis, pois cada AD individual pode ser alocada a uma CPU.

Random Forest (10/12)

RFs – aspectos positivos

- As RFs obtêm melhores resultados quando há **diversidade entre os classificadores base**.
 - Ou seja, de maneira ideal, deve existir baixa correlação entre as suas ADs.
- Estudos empíricos demonstraram que as RFs **não** são sensíveis ao número de atributos selecionados como candidatos a cada nó.
 - Para uma base com m atributos, tipicamente $\log_2(m + 1)$ costumam ser escolhidos.
- Um modelo RF tende a possuir acurácia muito superior a uma árvore de decisão gerada pelo CART.
 - Esta melhora na acurácia ocorre principalmente porque o modelo composto reduz a variância dos classificadores individuais.
- A **maior desvantagem** em relação ao CART é o fato de que uma RF não é diretamente interpretável (ao contrário do que ocorre com uma árvore de decisão simples).

Random Forest (11/12)

Hiperparâmetros na sklearn

- **n_estimators**: número de ADs do modelo (default = 100)
- **max_features**: número de atributos a serem considerados na hora de alocar um novo nó (default = sqrt)
 - O valor default corresponde à raiz quadrada do total de atributos da base.
- **bootstrap**: se o processo de bootstrap será utilizado (default = True)
 - Se for desabilitado, a base de treino será utilizada para gerar cada AD.

RandomForestClassifier

```
class sklearn.ensemble.RandomForestClassifier(n_estimators=100, *,
criterion='gini', max_depth=None, min_samples_split=2, min_samples_leaf=1,
min_weight_fraction_leaf=0.0, max_features='sqrt', max_leaf_nodes=None,
min_impurity_decrease=0.0, bootstrap=True, oob_score=False, n_jobs=None,
random_state=None, verbose=0, warm_start=False, class_weight=None,
ccp_alpha=0.0, max_samples=None, monotonic_cst=None)
```

Random Forest (12/12)

Outros tipos de Ensembles

- A técnica Random Forest é a mais conhecida dentre as técnicas para a construção de modelos compostos, mais conhecidos como **ensembles** (ou **ensemble models**).
- Porém existem algumas outras técnicas para o aprendizado de ensembles, como **Bagging Meta-Estimator**, **AdaBoost** e **Gradient Boosting**, entre outras.

Referências

- HAN, J.; KAMBER, M. PEI, J. Classification: Basic Concepts and Methods. *In*: HAN, J.; PEI, J.; TONG, H. **Data Mining: Concepts and Techniques**. 4. ed. Waltham: Morgan Kauffman, 2023. cap 6, p. 290 – 297.
- Decision Trees. *In*: scikit-learn documentation. Disponível em < <https://scikit-learn.org/stable/modules/tree.html> >. Acesso em: 04 set. 2026.
- DecisionTreeClassifier. *In*: scikit-learn documentation. Disponível em < <https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html> >. Acesso em: 04 set. 2026.
- RandomForestClassifier. *In*: scikit-learn documentation. Disponível em < <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html> >. Acesso em: 04 set. 2026.